

Claude 토큰 절약 8가지 방법 – Max 20x 플랜도 부족할 때 반드시 해야 할 것

3줄 요약

- 이 글은 토큰 절약이라는 관점에서 접근한 **컨텍스트 엔지니어링·하네스 엔지니어링** 이야기입니다
 - 작업에 필요한 최소한의 정보만 Claude에게 제공하는 설계가, 토큰 절약뿐 아니라 **Claude 아웃풋의 품질 향상**에도 기여합니다
 - Claude Max 20x 플랜에서도 토큰이 부족해진 필자의 시행착오를 소개합니다
-

시작하며

Claude Max 20x 플랜을 구독하고 있는데도, 생각보다 금방 토큰 사용 한도에 도달해버린다. 같은 고민을 가진 분들도 계실 것 같습니다.

필자는 Claude Code로 여러 에이전트를 병렬로 실행하면서 "투자 분석"과 "개인 프로젝트 개발"을 진행하고 있습니다.

투자 분석과 개인 개발을 여러 터미널에서 병렬로 돌리다 보면, 5시간 토큰 사용 한도를 2시간도 안 돼서 소진해버립니다.

이건 좀 심각합니다. 그렇다고 지갑 사정상 Anthropic에 더 많이 지출하는 것도 피하고 싶습니다.

그래서 **Claude의 토큰 소비를 줄이기 위해 시행착오를 겪은** 내용을 정리했습니다.

엄밀한 효과 측정은 하지 못했지만, 모든 방법을 적용한 결과, 현재로서는 5시간 사용 한도를 시간 내에 다 써버리는 일이 없어졌습니다.

전제: 무엇이 토큰을 잡아먹고 있었나

먼저 전제로, Claude의 토큰 소비는 "대화 본문"만으로 결정되지 않습니다.

실제로는 다음과 같은 것들이 토큰 소비 요인이 됩니다.

- 컨텍스트와 rules
- MCP 및 각종 툴 설정
- 길어진 세션 (= 대화 히스토리)
- 대량 파일 탐색 및 읽기
- JupyterNotebook이나 이미지처럼 용량이 커지기 쉬운 파일 조작
- agents / skills 정의의 장황한 설명

즉, 토큰 절약이란 **Claude의 컨텍스트 구성과 작업 플로우를 설계하는 것**이라고 봐야 합니다.

이는 토큰 절약뿐 아니라 Claude 아웃풋의 품질을 향상시키는 것으로도 이어집니다.

이번에 해본 것 8가지

요약

No.	방법	해결한 문제
1	<code>.claude/</code> 를 정리정돈했다	Claude 설정이 무거워 오버헤드가 증가하고 있었다
2	"압축형" 대화 규칙을 도입했다	불필요한 표현이 토큰을 낭비하고 있었다
3	jupytertext 로 Jupyter 조작을 경량화했다	Claude에게 JupyterNotebook은 무거운 작업이었다
4	세션 길이를 최적화했다	긴 세션을 이어가면서 토큰 소비가 누적되고 있었다
5	로컬 LLM으로 지식을 RAG화했다	Claude가 대량의 파일을 탐색하는 구성이 되어 있었다
6	에이전트 역할에 맞는 모델을 선택했다	단순 작업에 고성능 모델을 사용하고 있었다
7	sandbox 설정을 도입했다	Claude가 불필요하게 상위 디렉터리까지 탐색하는 동작이 있었다
8	<code>.claudeignore</code> 를 제대로 설정했다	읽지 않아도 될 파일을 읽고 있었다

1. `.claude/` 를 정리정돈했다

시험 삼아 넣어둔 채 방치했던 플러그인과 MCP를 검토해, 최소한으로 줄였습니다.

특히 루트 Claude 설정(`~/.claude/`)은 최대한 가볍게 하고, 공통으로 사용하는 것 외에는 각 작업 리포지터리 내의 설정으로 옮겼습니다.

`rules`는 자동으로 읽히기 때문에, 배치 위치와 `paths` 프론트매터 설계가 중요합니다.

```
---  
paths:  
  - "**.py"  
---  
# Python 개발 룰  
> .py 파일을 조작할 때만 읽히는 규칙을 여기에 작성
```

또한 agents / skills도 과감히 정리했습니다.

최소한 프론트매터를 정돈해, Claude가 매번 전부 읽지 않아도 되는 상태로 만들어 두는 것이 좋습니다.

이것만으로도, "일단 여러 가지를 Claude에 집어넣어 두는" 상태에서 크게 개선할 수 있습니다.

2. "압축형" 대화 규칙을 도입했다

[보완] 원문은 일본어 환경의 "原始人的 대화" 방식을 설명하고 있습니다. 한국어 환경에서 동등한 효과를 내도록 규칙을 한국어 문법 기준으로 재구성했습니다.

의미를 유지하면서 불필요한 표현을 제거하는 접근법입니다.

필자는 다음 내용을 모든 에이전트가 읽는 rules로 도입했습니다.

```
# 출력 토큰 압축
```

```
전체 출력에 적용.
```

```
## 삭제 대상
```

- 경어·존댓말 → 체언종지·용언종지 (입니다/합니다/습니다 금지)
- 완충어(음~/뭐/참고로/일단/기본적으로)
- 앞머리 인사(질문해 주셔서 감사합니다 등)
- 모호 표현(~일 수도 있습니다/~라고 생각됩니다/아마도)
- 장황한 표현(~할 수 있다 → ~가능)

- 장황한 접속(~하게 되므로 → 따라서)
- 자명한 조사(이/가/을/를/에서/는 - 의미 통하면 생략)
- 형식명사(설정을 변경하는 것 → 설정 변경)
- 보조동사(동작하고 있다 → 동작 중)
- 의미 중복(동의어 인접 반복 → 한쪽 삭제)
- 자명한 술어(문맥에서 추측 가능한 동사·형용사)
- 정보 부풀리기(질문받은 것만 답변. 포괄적 열거·보충·예시 코드 자발 생성 금지)

허용 기법

- 체언종지·용언종지
- 짧은 동의어 치환(대규모의 → 큰)
- 키워드 나열(조사 생략 후 공백 구분)
- 한자어 연결로 조사 생략(고부하 시 고속 → 고부하시고속)
- 순우리말→한자어화(빠르게 동작 → 고속동작)
- 격조사 생략 후 한자어 직결(Docker로 기동 → Docker기동)
- 접속 조사 → 화살표「→」대체

적용 제외

- 코드·커밋 메시지·PR 본문
- 사용자에게 파괴적 조작 확인 시(안전장치: 일반적인 정중한 한국어로 복귀)

또한 아웃풋뿐 아니라 다음과 같은 인풋도 마찬가지로 압축했습니다.

- 컨텍스트와 rules
- agents / skills 정의
- 설계서 등의 문서류

3. jupyter 로 Jupyter 조작을 경량화했다

투자 분석에서는 Jupyter Notebook(.ipynb)을 병렬로 생성하게 하는 경우가 많은데, 이것이 토큰을 대량으로 소비하고 있었습니다.

조사해 보니, `.ipynb` 를 그대로 다루는 것은 Claude에게 상당한 부담이 된다는 것을 알게 되었습니다.

`.ipynb` 내부에는 matplotlib 그래프 등이 base64로 인코딩된 이미지로 포함되는 경우가 많기 때문입니다.

그 때문에 파일 하나가 매우 커지기 쉽고, Claude Code는 세션 내에서 같은 파일을 반복 조작하기 때문에 소비가 점점 누적됩니다.

그래서 **Claude에게는 `.ipynb` 대신 `.py` 만 다루게 하는 구성**으로 변경했습니다.

jupytertext를 사용해 `.ipynb` 와 `.py` 를 동기화함으로써, 사람은 `.ipynb` 를, Claude는 `.py` 를 다루게 한 것입니다.

플로우는 다음과 같습니다.

1. Claude가 `.py` 를 편집한다
2. `jupytertext --sync` 로 `.ipynb` 에 반영한다
3. 사용자가 JupyterLab에서 실행·조회한다

2번 단계를 hooks로 자동화해, Claude가 `.py` 를 편집할 때마다 `.ipynb` 에 자동 반영되도록 했습니다.

이 환경에서는 1회 Jupyter 조작당 **"약 94% 토큰 절감"** 효과가 있다고 Claude가 평가했습니다. (정말일까요?)

효과는 Notebook의 내용에 따라 달라지겠지만, 작업 내용을 고려하면 상당한 효과가 있음은 분명합니다.

Claude에게 마크다운이나 코드 이외의 파일을 조작하게 하고 있다면, 불필요한 토큰을 낭비하고 있지는 않은지 의심해볼 가치가 있습니다.

[보완] 한국 개발 환경에서 JupyterLab은 로컬 실행 또는 AWS SageMaker Studio, Google Colab 등에서도 널리 활용됩니다. `jupytertext`는 `pip install jupytertext` 로 설치 후 `jupytertext --set-formats ipynb,py:percent notebook.ipynb` 로 동기화를 설정할 수 있습니다.

4. 세션 길이를 최적화했다

세션이 길어질수록, 과거의 모든 대화가 이후 비용에 영향을 미칩니다.

그 때문에 문맥이나 작업이 끊기는 지점에서는 적극적으로 `/clear` 를 실행해 **세션을 리셋** 하도록 했습니다.

또한 장기 플로우를 하나의 Skill로 실행하던 부분을 여러 Skill로 분할했습니다.

Skill 간에는 다음과 같은 **중간 산출물로 정보를 인계하고, 그 타이밍에 세션을 리셋**하는 설계로 했습니다.

- 작업 계획서
- 디베이트 서머리
- 다음 공정을 위한 인계 파일

작업 계획서 작성은 원래 작업 중 Claude의 기억이 희미해지는 것에 대한 대처로 도입한 방법이지만, 그 작성 타이밍에 Skill과 세션을 전환하는 설계를 추가함으로써 토큰 절약 면에서도 효과를 낼 수 있었습니다.

5. 로컬 LLM으로 지식을 RAG화했다

Claude에게 대량의 파일을 읽히지 않아도, 필요한 지식만 자연어로 끌어낼 수 있도록 로컬 LLM을 사용해 RAG를 구축했습니다.

목표는 **Claude가 탐색을 위해 대량의 파일을 읽는 상황을 줄이는** 것입니다.

예를 들어, 여러 문서에서 현재 설계를 파악하는 작업이 있다고 합시다.

이때 Claude에게 모든 문서를 읽히는 것이 아니라 다음과 같은 플로우로 처리합니다.

1. 사전에 로컬 LLM 쪽에서 문서를 벡터화해 둔다
2. Claude는 설계 파악에 필요한 질문만 로컬 LLM에 던진다
3. 로컬 LLM이 관련 문서를 검색해 답변한다
4. Claude는 그 답변을 바탕으로 설계를 파악한다

이렇게 하면 Claude가 매번 대량의 문서를 탐색할 필요가 없어집니다.

RAG뿐만 아니라 이런 단순한 작업을 로컬 LLM에게 위임하는 설계가 Claude의 토큰 절약에 효과적이라고 느끼고 있습니다.

[보완] 한국 환경에서 로컬 LLM 운영은 Apple Silicon Mac(M 시리즈), 또는 RTX 시리즈 GPU가 탑재된 Windows PC에서 Ollama + LangChain 조합이 많이 활용됩니다. 임베딩 모델로는 nomic-embed-text , 검색 백엔드로는 ChromaDB 또는 Qdrant 가 대표적인 선택지입니다.

6. 에이전트 역할에 맞는 모델을 선택했다

모든 작업에 고성능 모델을 사용할 필요는 없습니다.

단순한 포매팅, 분류, 요약, 검색 보조 같은 작업은 **경량 모델(Claude Haiku, Sonnet)**에 맡겨도 성능 차이가 크지 않습니다.

[보완] 현재(2025년 기준) Claude 모델 라인업에서 비용 대비 성능을 고려하면 다음 기준이 실용적입니다.

- **Claude Haiku:** 단순 분류·포매팅·라우팅 작업
- **Claude Sonnet:** 일반적인 코드 생성·요약·분석 작업
- **Claude Opus:** 복잡한 추론·설계 리뷰·멀티스텝 에이전트 오케스트레이션

7. sandbox 설정을 도입했다

이것은 원래 보안 관점에서 도입한 것이지만, 결과적으로 토큰 절약에도 효과가 있을 것 같아 소개합니다.

.claude/settings.json 에 sandbox 설정을 넣으면, Claude가 불필요하게 상위 디렉터리까지 탐색하려는 동작을 하지 않게 됩니다.

그 결과 탐색 범위가 좁아져, 불필요한 파일이나 컨텍스트에 접촉하기 어려워집니다.

[보완] `.claude/settings.json` 에서 `allowedDirectories` 또는 작업 범위를 명시적으로 설정하면 Claude가 프로젝트 루트 밖으로 나가지 않도록 제한할 수 있습니다. 보안과 토큰 절약을 동시에 달성하는 실용적인 방법입니다.

8. `.claudeignore` 를 제대로 설정했다

마지막은 기본이지만, 읽지 않아도 되는 파일을 `.claudeignore` 에 제대로 추가했습니다.

다음과 같이 용량이 크지만 읽을 의미가 없는 파일들은 처음부터 Claude의 시야에 들어오지 않도록 합시다.

- 대용량 데이터 파일 (CSV, Parquet, HDF5 등)
- 참조가 불필요한 빌드 산출물 (`dist/` , `build/` , `__pycache__/` 등)

[보완] `.claudeignore` 는 `.gitignore` 와 동일한 문법을 사용합니다. 이미 `.gitignore` 가 있다면 상당 부분 재활용할 수 있습니다. 다만 Claude가 참조해야 할 파일이 `.gitignore` 에 포함되어 있는 경우에는 별도로 `.claudeignore` 를 관리하는 것이 좋습니다.

정리

Claude의 토큰 절약을 위해 시작한 **필요 최소한의 정보만 제공하는 설계**는, 최근의 **컨텍스트 엔지니어링**·**하네스 엔지니어링 맥락**에서의 실천과 매우 유사한 것이 되었습니다.

Anthropic 공식 블로그에서는 AI 에이전트의 품질을 높이는 방법으로, 에이전트에게 작업에 필요한 정보만을 정확하게 제공하는 컨텍스트 설계의 중요성을 강조하고 있습니다.

[보완] Anthropic Engineering 블로그(<https://www.anthropic.com/engineering>)와 공식 문서(<https://docs.anthropic.com>)에서 Context Engineering 및 에이전트 설계 관련 최신 가이드를 확인할 것을 권장합니다.

Jaewoo Kim by @aiengineer